

Arquitectura de Sistemas Distribuidos de Alto Rendimiento: Optimización de APIs RESTful Hipermedia y Aplicaciones SPA

1. Introducción a los Paradigmas Arquitectónicos Modernos

La ingeniería de software contemporánea, particularmente en el ámbito de las aplicaciones web empresariales, ha experimentado un cisma fundamental en la forma en que se concibe la interacción entre el cliente y el servidor. Históricamente, el desarrollo en ecosistemas Java se basaba en la generación de contenido en el lado del servidor (*Server-Side Rendering*), utilizando tecnologías como JavaServer Pages (JSP) o motores de plantillas integrados en frameworks como Spring Web MVC. En este modelo tradicional, el servidor asumía una responsabilidad dual y monolítica: no solo gestionaba la lógica de negocio y la persistencia de datos, sino que también dictaba la presentación visual, construyendo documentos HTML completos que incluían la estructura, el estilo y los datos pre-renderizados.¹ La respuesta del servidor definía tanto el "qué" (la información) como el "cómo" (la presentación), acoplando fuertemente la interfaz de usuario con la infraestructura del backend.

Sin embargo, la evolución hacia arquitecturas orientadas a servicios y la demanda de experiencias de usuario más dinámicas y reactivas han precipitado la adopción del modelo de *Single Page Application* (SPA). En este paradigma, que constituye el núcleo del desafío técnico abordado en este informe, se produce una separación radical de responsabilidades. El backend se transforma en una entidad puramente proveedora de datos y lógica de negocio, exponiendo una API (Interfaz de Programación de Aplicaciones) que se adhiere, o aspira a adherirse, a los principios arquitectónicos REST (*Representational State Transfer*).¹

En este contexto, la aplicación que usted está desarrollando, compuesta por un backend en Java Spring Boot y un frontend en Vue.js con Vuetify, representa el estándar moderno de "Thick Client" o cliente pesado. A diferencia de los "Thin Clients" del pasado, donde el

navegador era un mero visualizador de HTML estático, su aplicación SPA transfiere una carga significativa de lógica computacional al dispositivo del usuario. La aplicación Vue.js no solo renderiza la interfaz, sino que gestiona el estado de la sesión, la navegación (enrutamiento), la validación de formularios y la orquestación de las llamadas a la red.¹

El desafío central que surge de esta arquitectura, y que motiva el presente análisis exhaustivo, es la tensión entre la pureza académica del diseño REST —específicamente el uso de Hipermedia como Motor del Estado de la Aplicación (HATEOAS)— y la eficiencia en el rendimiento de la red. Al optar por una API "100% REST" que utiliza URIs (*Uniform Resource Identifiers*) para representar claves foráneas en lugar de embeber datos, se introduce inevitablemente el problema de las "N+1 peticiones HTTP". Resolver este problema sin sacrificar la integridad semántica de REST requiere una implementación meticulosa de estrategias de *caching* (almacenamiento en caché) tanto en el servidor como en el cliente, así como una comprensión profunda de la negociación de contenido y el ciclo de vida de los recursos estáticos.¹

2. Fundamentos Teóricos y Prácticos del Diseño RESTful Purista

Es imperativo establecer una distinción clara entre una API HTTP genérica y una API verdaderamente RESTful, ya que muchas implementaciones industriales utilizan el término REST de manera laxa. Según la literatura técnica analizada, REST no es simplemente transmitir JSON sobre HTTP; es un estilo arquitectónico que impone restricciones específicas para garantizar la escalabilidad, la simplicidad y la independencia del despliegue.¹

2.1. Statelessness: La Eliminación del Estado de Sesión

Una de las piedras angulares de REST es la restricción de *statelessness* (sin estado). En las aplicaciones web tradicionales Java, el servidor mantenía el estado conversacional del usuario mediante objetos `HttpSession`, vinculados al cliente a través de cookies (típicamente `JSESSIONID`). Esto creaba una "mentira" conveniente sobre el protocolo HTTP, simulando una conexión persistente sobre un protocolo inherentemente sin conexión. Sin embargo, este enfoque presenta serios problemas de escalabilidad horizontal: si un servidor cae o si se necesita balancear la carga entre múltiples nodos, la replicación de sesiones se convierte en

un cuello de botella complejo y costoso.¹

En su arquitectura propuesta, la API debe ser completamente *stateless*. Esto significa que no existe el concepto de sesión en el servidor. Cada petición HTTP enviada desde el cliente Vue.js debe ser atómica y autocontenida, transportando toda la información necesaria para ser procesada, incluyendo las credenciales de autenticación y autorización. Esto elimina la dependencia de cookies de sesión y facilita que cualquier nodo del servidor pueda atender cualquier petición de manera independiente.¹

Autenticación Autocontenida: JWT vs. Cookies

Dado que no hay sesiones, la autenticación se maneja a través de metadatos en cada *request*. El mecanismo estándar para esto es el uso de tokens, específicamente JSON Web Tokens (JWT). A diferencia de una cookie opaca que solo es una referencia a un estado en memoria del servidor, un JWT es autocontenido: transporta la identidad del usuario y sus permisos (claims) firmados criptográficamente.

La autenticación no debe modelarse como un recurso (ej. `/login`), ya que "loguearse" es una acción, no una entidad. En su lugar, el intercambio de credenciales (usuario/password o tokens) se realiza mediante encabezados HTTP estándares. Inicialmente, se utiliza `Authorization: Basic base64(user:pass)` para obtener el token, y subsecuentemente `Authorization: Bearer {token}` para acceder a los recursos. Esto refuerza la naturaleza sin estado, ya que el servidor solo necesita verificar la firma del token sin consultar una base de datos de sesiones.¹

2.2. HATEOAS y la Navegabilidad del Grafo de Objetos

El Nivel 3 del Modelo de Madurez de Richardson, que define la gloria de REST, es HATEOAS. En este nivel, la API no solo devuelve datos, sino que guía al cliente sobre cómo interactuar con ella a través de hipervínculos. Las entidades están "hipervinculadas", formando un grafo de objetos navegable.

La premisa fundamental aquí es la distinción entre URL (*Uniform Resource Locator*) y URN (*Uniform Resource Name*). Una URN debe identificar de manera única y universal a un recurso. Esto tiene implicaciones profundas en el diseño de su API y es la causa raíz de su problema de rendimiento N+1.

El Principio de No Embeber Entidades

En una base de datos relacional, es trivial realizar un JOIN y traer la información de la dirección del usuario junto con sus datos básicos. En una API REST purista, esto se considera una mala práctica conocida como "embeber recursos". Si la entidad Address tiene su propia existencia y ciclo de vida, no debe incrustarse dentro del JSON de User.

Considere el siguiente escenario de violación de principios descrito en los documentos de investigación 1:

Si embebemos las direcciones dentro del usuario:

JSON

```
GET /users/1
{
  "username": "PAW",
  "addresses": [ { "street": "Calle Falsa 123" } ]
}
```

Esto crea ambigüedad en la actualización. Si un cliente envía un PUT /users/1 con "addresses": null, ¿significa que el usuario ya no tiene direcciones o que se han borrado las direcciones del sistema? Además, ahora la dirección vive en dos lugares: bajo /users/1 y potencialmente bajo su propia URI /addresses/5. Esto viola la unicidad de la URN y complica el control de acceso y el caché.¹

Por lo tanto, la solución correcta y estricta, que usted ha adoptado, es hipervincular:

JSON

```
GET /users/1
{
  "username": "PAW",
  "links": {
    "self": "/users/1",
    "addresses": "/users/1/addresses",
  }
}
```

```
"reportedIssues": "/issues?reportedBy=1"  
}  
}
```

Aquí, el ID de base de datos se vuelve irrelevante para el cliente; lo que importa es la URI. El cliente navega el grafo siguiendo estos enlaces. Sin embargo, esto precipita el problema N+1: para mostrar una lista de 50 usuarios con sus fotos de perfil (si la foto es un recurso separado), el cliente Vue debe realizar 1 petición para la lista y 50 peticiones adicionales para cada perfil.

2.3. Semántica de Verbos y Recursos

Para estructurar correctamente la API y facilitar la implementación del caching posterior, es vital adherirse a la semántica de los verbos HTTP. Los recursos son siempre sustantivos (/users, /posts), nunca acciones (/validateEmail, /login). Las operaciones se definen por el verbo HTTP utilizado.¹

Recurso (Sustantivo)	Verbo HTTP	Operación Semántica
/users	GET	Recuperar la lista completa de usuarios (paginada).
/users	POST	Crear un nuevo usuario. Retorna 201 Created + Header Location.
/users	PUT	Reemplazar la colección completa (raro, pero semánticamente posible). Idempotente.
/users	DELETE	Borrar la colección completa.
/users/{id}	GET	Recuperar un usuario específico.

/users/{id}	PUT	Reemplazar/Modificar el usuario. Idempotente.
/users/{id}	DELETE	Borrar el usuario.

El uso correcto de los códigos de estado es igualmente crítico para el control de flujo en el cliente Vue. Un 204 No Content es válido para una búsqueda que no arroja resultados pero fue exitosa, mientras que un 404 Not Found indica que el recurso solicitado no existe. Un 405 Method Not Allowed debe devolverse si se intenta un POST a un recurso de solo lectura. Esta riqueza semántica permite que el cliente tome decisiones inteligentes sin parsear mensajes de error arbitrarios.¹

3. Implementación Técnica en Backend: Transición a Jersey y JAX-RS

Para cumplir con los requisitos de una API REST estricta y facilitar la gestión de recursos que habilitará el caching, la arquitectura del backend debe migrar de los controladores tradicionales de Spring MVC hacia JAX-RS, utilizando Jersey como implementación de referencia. Esta decisión técnica no es trivial; Jersey ofrece un soporte más robusto y estandarizado para los principios HATEOAS y la negociación de contenido que el modelo MVC estándar.¹

3.1. Configuración del Contenedor de Servlets y Filtros

En una aplicación Spring Boot típica, el DispatcherServlet maneja todas las peticiones. Para integrar Jersey, debemos reconfigurar el web.xml (o la configuración programática equivalente) para eliminar el DispatcherServlet y registrar el ServletContainer de Jersey.

Un punto crucial para la coexistencia de la API y la SPA es registrar Jersey como un **Filtro** (Filter) y no como un Servlet. La razón técnica reside en la capacidad de los filtros para "dejar pasar" la petición. Al configurar el parámetro de inicialización jersey.config.servlet.filter.forwardOn404 en true, instruimos a Jersey a que, si una URI solicitada no coincide con ningún @Path definido en los recursos de la API, delegue el

procesamiento al siguiente eslabón en la cadena.¹

Esto es vital para servir los archivos estáticos de Vue.js. Si llega una petición a /assets/app.js, Jersey intentará buscar un recurso REST. Al no encontrarlo (404), en lugar de devolver un error, pasará la petición al contenedor de servlets por defecto (Jetty/Tomcat), que servirá el archivo estático. Esto elimina la necesidad de prefijos artificiales como /api/ en las URLs, permitiendo un espacio de nombres más limpio y semántico.¹

3.2. Data Transfer Objects (DTOs) y Mapeo de URIs

Para resolver el problema de la exposición de IDs internos y estructuras de base de datos, es obligatorio el uso de DTOs (*Data Transfer Objects*). Las entidades JPA (@Entity) nunca deben ser devueltas directamente por el controlador. El DTO actúa como un contrato de interfaz, desacoplando el modelo de dominio de su representación pública.

En su implementación, el DTO de User no debe tener un campo id de tipo Long o Integer visible. En su lugar, debe tener campos de tipo URI para sus relaciones, como self, reportedIssues, o assignedIssues.

Java

```
public class UserDto {  
    private String username;  
    private URI self;  
    private URI reportedIssues;  
    // Getters y Setters...  
}
```

Para poblar estos DTOs, se utiliza la inyección de dependencia de JAX-RS. El objeto UriInfo se inyecta en los métodos del controlador o en los constructores de los DTOs utilizando la anotación @Context. Este objeto provee constructores de URIs (UriBuilder) que permiten generar enlaces absolutos basados en el contexto actual de la petición, asegurando que los enlaces sean válidos independientemente del entorno de despliegue (localhost, dev, prod).¹

Java

```
public static UserDto fromUser(UriInfo uriInfo, User u) {  
    UserDto dto = new UserDto();  
    dto.setUsername(u.getUsername());  
    dto.setSelf(uriInfo.getBaseUriBuilder()  
        .path("users")  
        .path(String.valueOf(u.getId()))  
        .build());  
    return dto;  
}
```

3.3. Paginación vía Encabezados (RFC 5988)

Para mantener la pureza de la respuesta JSON (que debe ser una lista de usuarios [{}, {}] y no un objeto envuelto { data: [...], page: 1 }), la información de paginación se mueve a los encabezados HTTP. El estándar RFC 5988 define el encabezado Link para este propósito.

Cuando su cliente Vue solicita /users?page=2, el servidor responde con la lista de usuarios y un encabezado Link que contiene las URLs para las relaciones next, prev, first, y last. Esto permite que el cliente sea agnóstico de la lógica de construcción de URLs de paginación; simplemente sigue los enlaces proporcionados por el servidor.¹

3.4. Versionado mediante Negociación de Contenido

El versionado de la API es otro aspecto donde se debe evitar ensuciar la URL (ej. /v1/users). Una URN debe ser permanente. Si la representación del usuario cambia (ej. se divide name en firstName y lastName), el recurso sigue siendo el mismo usuario. Lo que cambia es la representación.

La solución arquitectónica correcta es utilizar **Vendor MIME-types** en los encabezados Accept y Content-Type. Se definen tipos personalizados como application/vnd.myapp.user.v1+json y application/vnd.myapp.user.v2+json. Los controladores de Jersey utilizan las anotaciones @Produces y @Consumes para enrutar la petición al método que sabe manejar esa estructura específica. Esto centraliza la complejidad del

versionado en la capa de entrada/salida, manteniendo el modelo de dominio limpio y único.¹

4. Resolución del Problema N+1 mediante Estrategias de Caching

El problema N+1 en su arquitectura no es un fallo de diseño, sino una consecuencia natural de desacoplar las entidades mediante hipervínculos. Dado que "embeber entidades no funciona" debido a los problemas de consistencia y atomicidad mencionados anteriormente ¹, la única solución viable que respeta los principios REST es un sistema de caching multinivel agresivo y bien configurado.

4.1. Caching Condicional (Backend)

El caching condicional es la primera línea de defensa para los datos dinámicos de la API (JSON). Su objetivo es reducir el ancho de banda y el tiempo de procesamiento, aunque no elimina necesariamente la latencia de red de ida y vuelta (Round Trip Time).

El mecanismo se basa en validadores: Last-Modified (basado en fechas) y ETag (Entity Tag, basado en hash del contenido).

1. **Flujo:** Cuando el cliente Vue solicita GET /users/1, el servidor calcula el hash del JSON resultante y lo envía en el encabezado ETag.
2. **Revalidación:** En la siguiente petición a la misma URI, el navegador (o el cliente HTTP configurado) envía automáticamente el encabezado If-None-Match con el valor del ETag recibido previamente.
3. **Respuesta 304:** El servidor compara el ETag recibido con el actual. Si el usuario no ha sido modificado, el servidor responde con un estado 304 Not Modified y un cuerpo vacío.

Esto es crítico para mitigar el N+1. Si su aplicación necesita cargar 50 usuarios, y 48 de ellos ya han sido visitados y cacheados por el navegador, esas 48 peticiones resultarán en respuestas 304 instantáneas de tamaño insignificante, reduciendo drásticamente la carga de transferencia de datos.¹ En Spring/Jersey, esto se implementa utilizando la clase Request de JAX-RS y el método evaluatePreconditions(EntityTag), o mediante filtros globales que calculan hashes MD5 de las respuestas.

4.2. Caching No Condicional (Recursos Estáticos)

Para los activos que componen su SPA (archivos JavaScript compilados, CSS, imágenes, fuentes), se debe aplicar una estrategia de caching no condicional. El objetivo aquí es que el navegador **nunca** contacte al servidor para verificar si el archivo ha cambiado.

El encabezado clave es Cache-Control. Para lograr la máxima eficiencia, se debe configurar el servidor para enviar:

Cache-Control: public, max-age=31536000, immutable

- public: Permite que cachés intermediarios (CDNs, proxies corporativos) almacenen el recurso.
- max-age=31536000: Indica que el recurso es válido por un año (31.536.000 segundos).
- immutable: Instruye al navegador que el contenido del archivo nunca cambiará, previniendo incluso las peticiones de revalidación cuando el usuario recarga la página manualmente.¹

Para implementar esto en Spring Boot, dado que estamos sirviendo los recursos estáticos a través del contenedor de servlets (gracias a la configuración de filtro de Jersey), necesitamos interceptar estas peticiones. La documentación sugiere la implementación de un UnconditionalCacheFilter personalizado. Este filtro debe mapear las URLs de recursos estáticos y agregar estos encabezados a la respuesta HttpServletResponse antes de que el contenido sea escrito.¹

4.3. El Desafío de la Invalidación: File Revving

La adopción de caching immutable introduce un problema obvio: ¿Cómo actualizamos la aplicación si los navegadores tienen cacheada la versión anterior por un año? La solución es el **File Revving** (Versionado de Archivos).

Esta técnica consiste en cambiar el nombre del archivo cada vez que su contenido se modifica. Esto se logra agregando un hash criptográfico del contenido al nombre del archivo.

- app.js -> app.a1b2c3d4.js
- styles.css -> styles.9x8y7z.css

Al cambiar el nombre del archivo, para el navegador se trata de un recurso completamente nuevo que no existe en su caché, forzando su descarga. Los archivos que no han cambiado

mantienen su hash y, por ende, su nombre, permitiendo que el navegador reutilice la versión cacheada.

Es fundamental entender que el archivo de entrada principal, usualmente `index.html`, **NO** debe ser cacheado incondicionalmente (o debe tener un tiempo de expiración muy corto, como no-cache), ya que es el único punto de entrada que contiene las referencias a los nuevos nombres de archivo con hash. Si el `index.html` se cacheara por un año, el usuario nunca recibiría las actualizaciones de la aplicación.¹

5. Arquitectura Frontend: Vue.js como Cliente Pesado (Thick Client)

La transición a una SPA implica que su aplicación Vue.js asume responsabilidades arquitectónicas complejas. Ya no es solo una capa de vista, sino una aplicación completa que se ejecuta en el navegador.

5.1. Estructura de Componentes y Servicios

Siguiendo las mejores prácticas de separación de responsabilidades (MVC en el cliente), se debe evitar colocar lógica de negocio o llamadas a la red directamente dentro de los componentes visuales (`.vue`).

- **Componentes (View):** Deben ser "tontos" y enfocados en la presentación. Reciben datos a través de props y emiten eventos. Su responsabilidad es renderizar la interfaz utilizando la librería de componentes Vuetify.¹
- **Servicios (Controller/Model):** Aquí reside la lógica de interacción con la API. Clases o módulos de TypeScript que encapsulan las llamadas axios o fetch. Estos servicios actúan como la capa de abstracción sobre la API REST.

5.2. Implementación de Caching en Cliente para Solucionar N+1

Aquí reside la solución táctica más efectiva para su problema de N+1 queries. Dado que el

backend envía URIs, el frontend debe ser inteligente en cómo las resuelve.

Patrón de Identity Map y Deduplicación de Peticiones

Cuando su componente de lista de usuarios itera sobre los resultados, encontrará múltiples referencias a las mismas entidades (ej. varios usuarios pueden tener el mismo rol o pertenecer al mismo departamento, referenciado por URI).

1. **Identity Map (Mapa de Identidad):** El servicio de frontend debe mantener un caché local (puede ser utilizando Pinia o un simple Map reactivo) donde las claves son las URIs y los valores son los objetos ya cargados. Antes de hacer una petición HTTP a una URI, el servicio verifica si ya existe en el mapa.
2. **Request Coalescing (Fusión de Peticiones):** Si múltiples componentes solicitan la misma URI simultáneamente, el servicio no debe lanzar múltiples peticiones de red. Debe almacenar la Promise de la petición en curso y retornarla a todos los solicitantes.

Ejemplo de Lógica de Servicio (Conceptual):

JavaScript

```
const resourceCache = new Map();
const inflightRequests = new Map();

function getResourceByUri(uri) {
  // 1. Verificar Caché
  if (resourceCache.has(uri)) {
    return Promise.resolve(resourceCache.get(uri));
  }
  // 2. Verificar Petición en Vuelo
  if (inflightRequests.has(uri)) {
    return inflightRequests.get(uri);
  }
  // 3. Realizar Petición
  const request = axios.get(uri).then(response => {
    resourceCache.set(uri, response.data);
    inflightRequests.delete(uri);
    return response.data;
  });
}
```

```
inflightRequests.set(uri, request);  
return request;  
}
```

Implementando este patrón, las "N" peticiones adicionales se reducen drásticamente. En una lista de 50 elementos, si todos apuntan a 3 recursos únicos repetidos, solo se realizarán 3 peticiones HTTP reales.

5.3. Gestión de Estado y Autenticación en SPA

La gestión del estado global de la aplicación (State Management) es vital en un "Thick Client". En el ecosistema Vue, esto se maneja típicamente con Pinia (sucesor de Vuex). El *store* actúa como la fuente de verdad, manteniendo los datos sincronizados entre diferentes componentes sin necesidad de volver a consultar al backend innecesariamente.¹

Para la autenticación, el token JWT recibido tras el login debe almacenarse en el cliente. Si bien el almacenamiento local (*localStorage*) es vulnerable a ataques XSS (Cross-Site Scripting), es el mecanismo estándar para arquitecturas sin estado que no usan cookies. Para mitigar riesgos, se debe asegurar que la aplicación sea robusta contra inyecciones de código. Los interceptores de axios deben configurarse para inyectar automáticamente el encabezado *Authorization: Bearer...* en cada petición saliente, y para manejar los errores 401 *Unauthorized* intentando un flujo de "Refresh Token" transparente para el usuario antes de redirigir al login.¹

6. Ciclo de Vida, Construcción y Optimización

El despliegue de una arquitectura híbrida Java/JS requiere un proceso de construcción unificado y optimizado.

6.1. Webpack y Minificación

Las herramientas modernas de construcción de frontend como Webpack (que subyace a Vue CLI) o Vite son esenciales para transformar el código fuente de desarrollo en artefactos de producción eficientes.

- **Minificación:** El proceso de construcción elimina espacios en blanco, comentarios y acorta nombres de variables en los archivos JS y CSS. Esto reduce significativamente el tamaño del *payload* inicial.
- **Tree Shaking:** Elimina el código muerto de las librerías que no se está utilizando (ej. importar solo los componentes de Vuetify necesarios en lugar de la librería completa).
- **Generación de Hashes:** Como se mencionó en la sección de caching, Webpack se encarga automáticamente del *file revving*, generando los nombres de archivo con hash e inyectándolos en el index.html.¹

6.2. Frontend Maven Plugin

Para integrar el flujo de trabajo de frontend (Node.js/NPM) con el ciclo de vida de construcción de Java (Maven), se recomienda utilizar el frontend-maven-plugin. Este plugin permite que el comando `mvn package` orqueste todo el proceso:

1. Descarga e instala una versión local de Node.js y NPM (asegurando reproducibilidad del entorno de build).
2. Ejecuta `npm install` para bajar dependencias del frontend.
3. Ejecuta `npm run build` para compilar y optimizar la SPA.
4. Copia los archivos resultantes (`dist/`) al directorio `src/main/resources/static` del artefacto Java.

Esto resulta en un único archivo WAR o JAR desplegable que contiene tanto el backend optimizado como el frontend minificado y listo para ser servido estáticamente por Spring/Jersey.¹

6.3. Auditoría con Lighthouse

Finalmente, la optimización no es un estado estático sino un proceso continuo. Se debe utilizar Google Lighthouse para auditar la aplicación. Esta herramienta evalúa métricas críticas como *First Contentful Paint* y *Time to Interactive*. Un reporte de Lighthouse permitirá verificar si la configuración de caching no condicional está funcionando (verificando los encabezados de respuesta de los archivos estáticos) y si la minificación y compresión (GZIP/Brotli) están activas en el servidor.¹

7. Conclusiones y Hoja de Ruta

La implementación de una API 100% RESTful con un cliente SPA moderno presenta desafíos arquitectónicos significativos, siendo el problema N+1 y la gestión del estado los más prominentes. Sin embargo, al adherirse a los patrones descritos en este informe, estos desafíos se transforman en ventajas de escalabilidad y mantenibilidad.

La hoja de ruta para su implementación debe ser:

1. **Backend:** Configurar Jersey como filtro, implementar DTOs con URLs inyectadas y mover la paginación a los headers Link.
2. **Infraestructura:** Implementar el UnconditionalCacheFilter en Spring para recursos estáticos y asegurar el soporte de ETags para la API.
3. **Frontend:** Construir una capa de servicios en Vue que implemente el patrón de *Identity Map* y *Request Coalescing* para resolver eficientemente las URLs provistas por el backend.
4. **Build:** Unificar los procesos con frontend-maven-plugin asegurando la minificación y el *file revving*.

Siguiendo esta estrategia, logrará una aplicación que no solo respeta la pureza académica de REST y HATEOAS, sino que ofrece una experiencia de usuario fluida y un rendimiento de red optimizado.

Obras citadas

1. NotasFinal.pdf